

Sender I/O: A Constructed Comparison

Document Number: P4053R0
Date: 2026-03-13
Audience: LEWG
Reply-to: Vinnie Falco vinnie.falco@gmail.com
Steve Gerbino steve@gerbino.co

Table of Contents

Abstract

Revision History

R0: March 2026 (post-Croydon mailing)

1. Disclosure

2. The Coroutine-Native Echo Server

3. Why I/O Results Are Different

4. Constructing the Sender Echo Server

5. “Just Use `set_value`”

What “Just Use `set_value`” Loses

6. “Just Split the Result”

What “Just Split the Result” Loses

7. “Just Use `set_error`”

What “Just Use `set_error`” Loses

8. “Just Decompose It”

What “Just Decompose It” Loses

9. The Trade-Off

When the Byte Count Determines Correctness

10. Qt Sender vs. Coroutine

11. Structured Concurrency

12. Frequently Raised Concerns

Q1: Is the echo server too minimal to be representative?

Q2: Are these gaps being addressed by ongoing work?

Q3: Could the byte count be stored in the operation state instead of shared state?

Q4: Could the entire compound result be sent through `set_error` ?

Q5: Do coroutines provide structured concurrency?

Q6: Is the invitation constructed to be unanswerable?

Q7: Does “just decompose it” work if the return-type constraint is solved?

Q8: What does the coroutine-native approach lose?

Q9: Does the coroutine-native model provide composition?

Q10: Does the sender model compose across execution contexts in ways coroutines cannot?

13. Invitation

14. Acknowledgments

References

Abstract

Four sender-based TCP echo servers are constructed from [P2300R10](#)^[1] and [P3552R3](#)^[2] and compared against a coroutine-native echo server. No construction achieves both full data preservation and channel-based composition without shared state or exceptions. The three-channel model is correct for infrastructure operations with binary outcomes. The finding is about domain, not defect.

This paper is one of a suite of five that examines the relationship between compound I/O results and the sender three-channel model. The companion papers are [D4050R0](#)^[15], “On Task Type Diversity”; [D4054R0](#)^[7], “Two Error Models”; [D4055R0](#)^[13], “Consuming Senders from Coroutine-Native Code”; and [D4056R0](#)^[14], “Producing Senders from Coroutine-Native Code.”

Revision History

R0: March 2026 (post-Croydon mailing)

- Initial version.

1. Disclosure

The authors developed and maintain [Corosio](#)^[3] and [Capy](#)^[4] and believe coroutine-native I/O is the correct foundation for networking in C++. The sender model provides compile-time work graphs, lazy pipeline evaluation, and generic algorithms over heterogeneous sender types that the coroutine-native model does not. These are genuine strengths. The findings in this paper are limited to compound I/O results and hold regardless of which library implements the coroutine-native layer. The authors provide information, ask nothing, and serve at the pleasure of the chair.

The authors regard `std::execution` as an important contribution to C++ and support its standardization for the domains it serves well - GPU dispatch, heterogeneous execution, and compile-time work-graph composition among them. Nothing in this paper or its companions argues for removing, delaying, or diminishing `std::execution`. The authors' position is narrower: that networking and stream I/O present a compound-result structure that the three-channel model was not designed to carry, and that this domain is better served by a coroutine-native facility that can coexist with senders and interoperate where the domains meet. Two models, each correct for its domain, is a stronger standard than one model asked to serve both.

2. The Coroutine-Native Echo Server

[Corosio](#)^[3] `do_session` :

```
copy::task<> do_session()
{
    for (;;)
    {
        auto [ec, n] = co_await sock_.read_some(
            copy::mutable_buffer(
                buf_, sizeof buf_));

        auto [wec, wn] = co_await copy::write(
            sock_, copy::const_buffer(buf_, n));

        if (ec || wec)
            break;
    }
    sock_.close();
}
```

Both values visible. No exceptions.

3. Why I/O Results Are Different

Infrastructure operations have binary outcomes:

Operation	Success	Failure
<code>malloc</code>	Block returned	Allocation failed
<code>fopen</code>	File handle returned	Open failed
<code>pthread_create</code>	Thread running	Creation failed
GPU kernel launch	Kernel running	Launch failed
Timer arm	Timer armed	Resource limit

Every row is binary. The three channels map unambiguously.

I/O operations return compound results:

Operation	Result
<code>read</code>	<code>(status, bytes_transferred)</code>
<code>write</code>	<code>(status, bytes_written)</code>

Every OS delivers both values together.^[8] Chris Kohlhoff identified the consequence for senders in [P2430R0](#)^[5]:

“Due to the limitations of the `set_error` channel (which has a single ‘error’ argument) and `set_done` channel (which takes no arguments), partial results must be communicated down the `set_value` channel.”

Four approaches follow.

4. Constructing the Sender Echo Server

Each approach is constructed from [P2300R10](#)^[1] and [P3552R3](#)^[2].

Kühl enumerated five channel-routing options in [P2762R2](#)^[10] Section 4.2 and noted: “some of the error cases may have been partial successes...using the `set_error` channel taking just one argument is somewhat limiting.” Shoop identified the same difficulty in [P2471R1](#)^[11]: completion tokens translating to senders “must use a heuristic to type-match the first arg.”

[P2300R10](#)^[1] Section 1.3 demonstrates the pattern:

```
return read_socket_async(socket, span{buff})
| execution::let_value(
    [](error_code err,
       size_t bytes_read) {
        if (err != 0) {
            // partial success
        } else {
            // full success
        }
    }
);
```

“Just use `set_value`” piped into a `let_value` decomposition. The example does not show what happens to `bytes_read` when the error must reach `set_error`.

5. “Just Use `set_value`”

The I/O sender calls `set_value(error_code, size_t)` for all outcomes - route everything through the value channel. This paper calls this pattern “just use `set_value`.” Structured bindings work ([P3552R3](#)^[2] Section 4.4):

```

auto do_session(auto& sock, auto& buf)
-> std::execution::task<void>
{
    for (;;)
    {
        auto [ec, n] = co_await async_read(
            sock, net::buffer(buf));

        auto [wec, wn] = co_await async_write(
            sock, net::const_buffer(buf, n));

        if (ec || wec)
            break;
    }
}

```

Nearly identical to Corosio. Both values visible. No exceptions. No sender composition.

What “Just Use `set_value`” Loses

The I/O sender never calls `set_error`. The composition algebra is bypassed:

- `when_all` does not cancel siblings on I/O failure - the failure arrived through `set_value`.
- `upon_error` is unreachable.
- `retry` does not fire.

An HTTP/2 multiplexer issuing concurrent reads via `when_all` expects sibling cancellation on stream failure. Under “just use `set_value`,” the failure arrives through `set_value` and `when_all` continues reading on dead streams.

The model reduces to one channel. `set_error` serves no purpose for I/O senders under “just use `set_value`.” If this is the correct approach, then the composition algebra that justifies the sender model’s complexity over coroutines does not apply to the I/O domain.

6. “Just Split the Result”

Route the error code through `set_error`; capture the byte count in shared state. This paper calls this pattern “just split the result.”

```

auto do_session(auto& sock, auto& buf)
-> std::execution::task<void>
{
    for (;;)
    {
        std::size_t n = 0;

        co_await (
            async_read(sock, net::buffer(buf))
            | then([&](std::size_t bytes) {
                n = bytes;
            })
            | upon_error(
                [&](std::error_code ec) {
                    // ec visible, n stale
                }));

        co_await (
            async_write(
                sock, net::const_buffer(buf, n))
            | then([&](std::size_t bytes) {
                n = bytes;
            })
            | upon_error(
                [&](std::error_code ec) {
                    // ec visible, n stale
                }));

    }
}

```

All three channels in use. `upon_error` reachable. `when_all` cancels siblings. `retry` fires.

What “Just Split the Result” Loses

The byte count bypasses the channels:

- `retry` fires on `set_error` but the byte count is in `n`, not in the error channel.
- `upon_error` receives `error_code` alone. `n` reflects the previous stage, not the failed one.
- Shared mutable state across continuation boundaries.

“Just use `set_value`” with the error code moved to `set_error` and the byte count moved to shared state.

7. “Just Use `set_error`”

`set_value(size_t)` on success, `set_error(error_code)` on failure. This paper calls this pattern “just use `set_error`.” P3552R3^[2] converts `set_error` to an exception via `AS-EXCEPT-PTR` (the exposition-only function that converts `set_error` arguments to `exception_ptr`). No non-throwing path:

```
auto do_session(auto& sock, auto& buf)
-> std::execution::task<void>
{
    try {
        for (;;)
        {
            auto n = co_await async_read(
                sock, net::buffer(buf));

            co_await async_write(
                sock, net::const_buffer(buf, n));
        }
    } catch (std::system_error const& e) {
        // ECONNRESET, EPIPE, EOF arrive here
    }
}
```

What “Just Use `set_error`” Loses

- Byte count. 500 of 1,000 bytes written before `ECONNRESET` - gone.
- Non-throwing path. Every `ECONNRESET` requires `make_exception_ptr` + `rethrow_exception`.
- Visible error path. The error hides in `catch`, separated from the `co_await` site.

8. “Just Decompose It”

Pipe “just use `set_value`” into `let_value`, inspect both values, re-route the error code to `set_error`. This paper calls this pattern “just decompose it.”


```
#include <exec/variant_sender.hpp> // stdexec

async_read(sock, net::buffer(buf))
| let_value(
    [](std::error_code ec, std::size_t n) {
        auto succ = [&] {
            return just(n); };
        auto fail = [&] {
            return just_error(ec); };
        using result =
            exec::variant_sender<
                decltype(succ()),
                decltype(fail())>;
        if (ec)
            return result(fail());
        return result(succ());
    })
| upon_error([](std::error_code ec) {
    // reachable
});
```

`exec::variant_sender` is from `stdexec`, not P2300R10^[1]. This construction uses a facility the invitation (Section 13) does not permit. Even with `variant_sender` standardized, the data-loss problem persists: `just_error(ec)` carries only the error code (Q7). The return-type constraint and the data-loss constraint are independent.

The handler sees both values. Downstream, `upon_error` is reachable, `when_all` cancels siblings, `retry` fires.

What “Just Decompose It” Loses

`just_error(ec)` carries only the error code. `set_error` takes a single argument (P2300R10^[1] [exec.recv.concepts]).

- `retry` never sees the byte count.
- `upon_error` receives `error_code` alone.
- Partial write. 500 of 1,000 bytes before `ECONNRESET` - gone once `just_error` emits.

“Just decompose it” is a hybrid of “just use `set_value`” and “just use `set_error`.” The byte count is still lost on the error path.

9. The Trade-Off

Each sender approach pays a cost the coroutine-native approach does not.

Property	"Just use <code>set_value</code> "	"Just split the result"	"Just use <code>set_error</code> "	"Just decompose it"	Corosio
Data preservation					
Partial write preserved	Yes	Yes	No	No (on error path)	Yes
Byte count in completion sig	Yes	No (side state)	No (discarded)	No (on error path)	Yes (return value)
Retry sees byte count	No	No	No	No	Yes
Composition					
<code>when_all</code> cancels on I/O error	No	Yes	Yes	Yes	Yes (value-based)
Error handler reachable	No (<code>upon_error</code>)	Yes (<code>upon_error</code>)	Yes (<code>upon_error</code>)	Yes (<code>upon_error</code>)	Yes (<code>if (ec)</code>)
Retry on I/O error	No (<code>retry</code>)	Yes (<code>retry</code>)	Yes (<code>retry</code>)	Yes (<code>retry</code>)	Yes (<code>retry_after</code>)
Channels used for I/O	1 of 3	3 of 3	3 of 3	3 of 3	Values (no channels)
Compile-time work graph	Yes (lazy)	Yes (lazy)	Yes (lazy)	Yes (lazy)	No
Ergonomics					
Error code at call site	Yes	Yes	No (in <code>catch</code>)	Yes (in handler)	Yes
Byte count at call site	Yes	Yes	No (discarded)	Yes (in handler)	Yes

Property	"Just use <code>set_value</code> "	"Just split the result"	"Just use <code>set_error</code> "	"Just decompose it"	Corosio
Exception on <code>ECONNRESET</code>	No	No	Yes	No	No
Shared mutable state required	No	Yes	No	No	No

Infrastructure operations face no such trade-off. Their outcomes are binary. A retry policy that distinguishes zero-progress failures from partial-progress failures needs the byte count - retrying a read that transferred zero bytes but not one that stalled after partial transfer.

When the Byte Count Determines Correctness

Many HTTP servers - including Google's - skip TLS `close_notify`. The composed read returns `(stream_truncated, n)`. If `n` equals `Content-Length`, the body is complete and the truncation is harmless. If `n` is less, the body is incomplete. The byte count determines correctness.

Coroutine-native:

```
auto [ec, n] = co_await tls_read(
    stream, body_buf);
if (ec == stream_truncated
    && n == content_length)
    ec = {}; // body complete, ignore truncation
```

"Just use `set_error`":

```
try {
    auto n = co_await tls_read(
        stream, body_buf);
} catch (std::system_error const& e) {
    // stream_truncated arrives here
    // n is gone - cannot check content_length
}
```

Under "just decompose it," the `let_value` handler sees both values - but only if it has the HTTP framing context. A generic TLS adapter does not. Once `just_error(stream_truncated)` emits, the byte count is gone.

This boundary is the **abstraction floor**:

Region	What the code sees
Above the floor	<code>error_code</code> alone - composition works
Below the floor	<code>(error_code, size_t)</code> - both values intact

The coroutine-native model has no such boundary.

An HTTP/2 multiplexer issuing concurrent reads via `when_all` faces the same table at every I/O boundary.

10. Qt Sender vs. Coroutine

Ville Voutilainen's `libunifex-with-qt`^[16] contains a chunked HTTP downloader written as both a sender pipeline and a coroutine.

Sender pipeline - “Just split the result”:

```

auto repeated_get =
    let_value(
        just(req)
        | then([this](auto request) {
            return setupRequest(request,
                                bytesDownloaded, chunkSize);
        }),
        [this](auto request) {
            auto* reply = nam.get(request);
            return QObjectAsSender(reply,
                                    &QNetworkReply::finished)
                | then([reply] {
                    return reply;
                });
        })
    | then([this](auto* reply) {
        bytesDownloaded += // shared state
            reply->contentLength();
        reply->deleteLater();
    })
    | then([this] {
        return bytesDownloaded // shared state
            == contentLength;
    })
    | repeat_effect_until();

```

Coroutine - both values visible:

```

exec::task<void> doFetchWithCoro()
{
    while (bytesDownloaded != contentLength)
    {
        req = setupRequest(req,
                            bytesDownloaded, chunkSize);
        auto* reply = nam.get(req);
        co_await QObjectAsSender(reply,
                                &QNetworkReply::finished);
        bytesDownloaded +=
            reply->contentLength();
        reply->deleteLater();
    }
}

```

Voutilainen's example is a demonstration of Qt/stdexec integration, not production code, and the authors are grateful for it - it is the only published side-by-side comparison of a sender pipeline and a coroutine performing the same work. The sender pipeline is "just split the result" in the trade-off table. The byte count is in `this->bytesDownloaded`, not in any channel. `repeat_effect_until` reads the data member directly. The coroutine version accesses the same data member, but the coroutine's sequential execution - guaranteed by the executor - serializes access to `bytesDownloaded`. The sender pipeline's lambda captures create aliasing across continuation boundaries that the type system does not prevent.

11. Structured Concurrency

`Capy`^[4] provides `when_all` and `when_any` as coroutine-native primitives with structured cancellation: child operations complete before the parent continues, and stop tokens propagate through `io_env`.

```
copy::task<dashboard> load_dashboard(
    std::uint64_t user_id)
{
    auto [name, orders, balance] =
        co_await copy::when_all(
            fetch_user_name(user_id),
            fetch_order_count(user_id),
            fetch_account_balance(user_id));
    co_return dashboard{name, orders, balance};
}
```

```
copy::task<> timeout_a_worker()
{
    auto result = co_await copy::when_any(
        background_worker("worker"),
        copy::delay(100ms));

    if (result.index() == 1)
        std::cout << "Timeout fired\n";
}
```

Both models provide structured concurrency; the sender `when_all` additionally provides compile-time work-graph visibility that the coroutine `when_all` does not (Q8, Q10). They differ in where the compound result is visible when the composition decision is made. The trade-off table (Section 9) applies at every I/O boundary

inside a structured concurrency scope.

12. Frequently Raised Concerns

Q1: Is the echo server too minimal to be representative?

The echo server is deliberately minimal. The compound-result problem is per-operation: adding protocol complexity adds more call sites with the same trade-off, not a different one. The sender model's composition strengths are orthogonal to the channel-routing decision each I/O operation must make. If the committee believes a more complex pipeline would change the finding, we invite the sender community to provide one. We will construct the comparison and publish the results.

Q2: Are these gaps being addressed by ongoing work?

Several of the ergonomic issues documented here are the subject of active committee work, including [P3796R1](#)^[17] and [D3980R0](#)^[18]. The authors welcome that work. However, the trade-off documented in Section 9 is structural at two levels. First, `set_error` takes a single argument; no iteration of the sender algorithms can deliver both the error code and the byte count through `set_error` without changing the channel model itself. Second, even if `set_error` accepted compound results, the I/O sender must still choose which channel to call - and that choice is context-dependent. `ECONNRESET` is fatal in an HTTP request handler but expected in a long-polling connection. The classification that determines the channel is application logic, not I/O logic. No context-free channel assignment is correct for all protocols. Ergonomic improvements to `let_value`, `variant_sender`, or `task` do not alter either constraint.

Q3: Could the byte count be stored in the operation state instead of shared state?

A variant of "just split the result" stores the byte count in the operation state rather than a local variable, scoping the lifetime to the operation. This localizes the lifetime hazard and is a meaningful improvement. The structural problem is unchanged: the byte count remains outside the completion signature, invisible to `retry`, `upon_error`, and every generic algorithm that operates on channel data.

Q4: Could the entire compound result be sent through `set_error` ?

Sending `tuple<error_code, size_t>` through `set_error` preserves both values but changes the completion signatures expected by generic sender algorithms. `retry` fires on a tuple. `upon_error` receives application data. That is a change to the channel model. If the committee wishes to pursue that direction, it deserves its own paper and its own design review.

Q5: Do coroutines provide structured concurrency?

Yes. See Section 11.

Q6: Is the invitation constructed to be unanswerable?

The invitation asks whether the three-channel model can represent compound I/O results without loss, using the model's own facilities and semantics. The constraints - preserve data, retain composition, do not alter channel semantics, use specified facilities - are the properties the model claims to provide. If no construction can satisfy all of them simultaneously, that is the finding. The authors will incorporate any construction that does and re-evaluate every finding in this paper.

Q7: Does “just decompose it” work if the return-type constraint is solved?

`let_value` requires its callable to return a single sender type. `just(n)` and `just_error(ec)` are different types. Section 8 solves this with `exec::variant_sender` (`stdexec`). Suppose the return-type constraint is solved in the standard (for example, by a future `variant_sender`). The byte count is still lost: `just_error(ec)` carries only the error code. `set_error` takes a single argument. Solving the return-type problem does not solve the data-loss problem.

Q8: What does the coroutine-native approach lose?

Compile-time work graphs, lazy pipeline evaluation without coroutine frames, and generic algorithms over heterogeneous sender types. These are real costs. They are the sender model's strengths. This paper does not argue that coroutines replace senders. It argues that compound I/O results do not fit three mutually exclusive channels. The sender model serves its domain. The finding is about domain, not defect.

Q9: Does the coroutine-native model provide composition?

`when_all` and `when_any` (Section 11). Both values visible inside the combinator. The sender `when_all` cannot see the byte count because the channel split already happened. The coroutine `when_all` can, because the coroutine body inspects `(ec, n)` before the composition decision is made. Both models provide structured concurrency. They differ in where the compound result is visible.

Q10: Does the sender model compose across execution contexts in ways coroutines cannot?

Yes. Compile-time work graphs connecting GPU dispatch, thread pool submission, and event loop scheduling in a single statically typed pipeline - that is the sender model's domain and its genuine strength. Q8 acknowledges the cost. The compound-result problem is orthogonal. A GPU kernel launch is a binary outcome. A `read` is not.

The sender model's cross-context composition works because its target operations are infrastructure operations with binary outcomes. The finding in this paper is about the operations that are not binary. This paper identifies the domain boundary.

13. Invitation

Construct a sender-based echo server that:

- preserves compound I/O results on the error path,
- retains channel-based composition (`when_all` , `upon_error` , `retry`),
- does not alter the completion signatures expected by generic sender algorithms (`retry` , `upon_error` , `when_all`) from those algorithms' specified behavior,
- avoids exception round-trips for routine error codes, and
- uses only the algorithms and sender adapters specified in P2300R10^[1], P3552R3^[2], and P3149R9^[6].

If the committee believes these five properties should not be expected simultaneously for I/O senders, that finding is equally valuable - it identifies the domain boundary.

The authors will incorporate any such construction and re-evaluate every finding.

14. Acknowledgments

The authors thank Dietmar Kühl for the channel-routing enumeration in P2762R2^[10] and for `beman::execution` , Chris Kohlhoff for identifying the partial-success problem in P2430R0^[5], Kirk Shoop for the completion-token heuristic analysis in P2471R1^[11], Peter Dimov for the refined channel mapping in P4007R0^[9], "Senders and Coroutines," Michał Dominiak, Eric Niebler, and Lewis Baker for `std::execution` , Ian Petersen, Jessica Wong, and Kirk Shoop for `async_scope` , and Fabio Fracassi for P3570R2^[12], "Optional variants in sender/receiver." The authors also thank Ville Voutilainen and Jens Maurer for reflector discussion, and Herb Sutter for identifying the need for constructed comparisons.

References

1. P2300R10 - "std::execution" (Michał Dominiak et al., 2024). <https://wg21.link/p2300r10>

2. **P3552R3** - "Add a Coroutine Task Type" (Dietmar Kühl, Maikel Nadolski, 2025). <https://wg21.link/p3552r3>
3. **cppalliance/corosio** - Coroutine-native networking library. <https://github.com/cppalliance/corosio>
4. **cppalliance/capy** - Coroutine I/O primitives library. <https://github.com/cppalliance/capy>
5. **P2430R0** - "Partial success scenarios with P2300" (Chris Kohlhoff, 2021). <https://wg21.link/p2430r0>
6. **P3149R9** - "`async_scope` - Creating scopes for non-sequential concurrency" (Ian Petersen, Jessica Wong, Kirk Shoop, et al., 2025). <https://wg21.link/p3149r9>
7. **D4054R0** - "Two Error Models" (Vinnie Falco, 2026). <https://wg21.link/d4054r0>
8. IEEE Std 1003.1-2024 - POSIX `read()` / `write()` specification.
<https://pubs.opengroup.org/onlinepubs/9799919799/>
9. **P4007R0** - "Senders and Coroutines" (Vinnie Falco, Mungo Gill, 2026). <https://wg21.link/p4007r0>
10. **P2762R2** - "Sender/Receiver Interface For Networking" (Dietmar Kühl, 2023). <https://wg21.link/p2762r2>
11. **P2471R1** - "NetTS, ASIO and Sender Library Design Comparison" (Kirk Shoop, 2021).
<https://wg21.link/p2471r1>
12. **P3570R2** - "Optional variants in sender/receiver" (Fabio Fracassi, 2025). <https://wg21.link/p3570r2>
13. **D4055R0** - "Consuming Senders from Coroutine-Native Code" (Vinnie Falco, Steve Gerbino, 2026).
<https://wg21.link/d4055r0>
14. **D4056R0** - "Producing Senders from Coroutine-Native Code" (Vinnie Falco, Steve Gerbino, 2026).
<https://wg21.link/d4056r0>
15. **D4050R0** - "On Task Type Diversity" (Vinnie Falco, 2026). <https://wg21.link/d4050r0>
16. Ville Voutilainen, **libunifex-with-qt** - Qt/stdexec integration examples (2024).
<https://git.qt.io/vivoutil/libunifex-with-qt>
17. **P3796R1** - "Coroutine Task Issues" (Dietmar Kühl, 2025). <https://wg21.link/p3796r1>
18. **D3980R0** - "Task's Allocator Use" (Dietmar Kühl, 2026). <https://wg21.link/d3980r0>